

Taller: diseño e implementación de un algoritmo genético

Introducción a la Inteligencia Artificial

Estudiante(s): _____ **Fecha:** _____
Modalidad: Trabajo en parejas **Valor:** 5,0 puntos
Herramienta: Python **Duración sugerida:** 4 horas

Propósito

Diseñar, implementar y evaluar un algoritmo genético para resolver un problema de optimización combinatoria, identificando sus componentes principales: representación de individuos, población inicial, función de aptitud, selección, cruce, mutación, elitismo y criterio de terminación.

Situación problema: selección óptima de proyectos

Una organización dispone de un presupuesto limitado para financiar proyectos de innovación. Cada proyecto requiere una inversión y proporciona un beneficio estimado. El objetivo consiste en seleccionar el conjunto de proyectos que produzca el mayor beneficio posible sin superar el presupuesto disponible.

Proyecto	Costo	Beneficio
P1	12	24
P2	7	13
P3	11	23
P4	8	15
P5	9	16
P6	14	28
P7	6	11
P8	10	19
P9	5	9
P10	13	25

El presupuesto máximo disponible es de **50 unidades monetarias**. Cada proyecto puede seleccionarse como máximo una vez.

Representación de una solución

Cada individuo se representa mediante un cromosoma binario de diez genes:

$$X = (x_1, x_2, \dots, x_{10}), \quad x_i = \begin{cases} 1, & \text{si el proyecto } i \text{ es seleccionado,} \\ 0, & \text{si el proyecto } i \text{ no es seleccionado.} \end{cases}$$

Por ejemplo, el individuo

$$X = (1, 0, 1, 0, 0, 1, 0, 1, 0, 0)$$

representa la selección de los proyectos P1, P3, P6 y P8. El costo y el beneficio se calculan mediante:

$$C(X) = \sum_{i=1}^{10} c_i x_i, \quad B(X) = \sum_{i=1}^{10} b_i x_i.$$

El algoritmo debe maximizar $B(X)$, sujeto a la restricción $C(X) \leq 50$.

Punto 1. Formulación del problema

Describa formalmente el problema de optimización y responda:

- a) ¿Cuál es la variable de decisión?
- b) ¿Cuál es la función objetivo?
- c) ¿Cuál es la restricción principal?
- d) ¿Cómo se representa un individuo?
- e) ¿Cuántas soluciones diferentes existen en el espacio de búsqueda?

Calcule manualmente el costo y el beneficio del individuo

$$X = (1, 0, 1, 1, 0, 0, 1, 0, 1, 0)$$

y determine si representa una solución válida.

Punto 2. Población y función de aptitud

Implemente una función que genere una población inicial aleatoria de cromosomas binarios. El tamaño de la población debe ser configurable; utilice inicialmente $N = 20$.

Diseñe una función de aptitud que premie las soluciones con mayor beneficio y penalice aquellas que excedan el presupuesto. Puede emplear:

$$\text{fitness}(X) = \begin{cases} B(X), & \text{si } C(X) \leq 50, \\ B(X) - \lambda(C(X) - 50), & \text{si } C(X) > 50. \end{cases}$$

Utilice inicialmente $\lambda = 5$. El programa debe mostrar para cada individuo:

- cromosoma y proyectos seleccionados;
- costo total y beneficio total;
- valor de aptitud;
- indicación de si la solución es válida.

Explique qué sucede si el valor de λ es demasiado pequeño o demasiado grande.

Punto 3. Operadores genéticos

Implemente los siguientes operadores:

Selección

Utilice selección por torneo. Seleccione aleatoriamente tres individuos y escoja como padre al individuo con mayor aptitud.

Cruce

Implemente cruce de un punto. Seleccione una posición aleatoria e intercambie los segmentos de los padres. Por ejemplo:

$$\begin{aligned} P_1 &= (1, 0, 1, 1 \mid 0, 0, 1, 0, 1, 0), \\ P_2 &= (0, 1, 0, 0 \mid 1, 1, 0, 1, 0, 1), \\ H_1 &= (1, 0, 1, 1 \mid 1, 1, 0, 1, 0, 1), \\ H_2 &= (0, 1, 0, 0 \mid 0, 0, 1, 0, 1, 0). \end{aligned}$$

Mutación

Implemente una mutación binaria. Cada gen podrá cambiar de 0 a 1 o de 1 a 0 con una probabilidad p_m . Utilice inicialmente $p_m = 0,05$.

Realice manualmente un ejemplo completo de selección, cruce y mutación con dos individuos de la población inicial. Registre los padres, el punto de cruce, los descendientes antes de la mutación y los descendientes finales.

Punto 4. Algoritmo genético completo [1,0 puntos]

Integre los componentes anteriores en un algoritmo genético que ejecute el siguiente procedimiento:

1. Generar la población inicial.
2. Evaluar la aptitud de todos los individuos.
3. Seleccionar los padres.
4. Aplicar los operadores de cruce y mutación.
5. Construir la nueva población.
6. Conservar el mejor individuo mediante elitismo.
7. Repetir el proceso hasta alcanzar el criterio de terminación.

Utilice como configuración inicial:

Parámetro	Valor
Tamaño de población	20
Número de generaciones	100
Probabilidad de cruce	0,80
Probabilidad de mutación	0,05
Tamaño del torneo	3
Número de individuos elite	1
Penalización λ	5

El programa debe registrar en cada generación el número de generación, la mejor aptitud, el beneficio y costo de la mejor solución, su cromosoma y la aptitud promedio de la población. Al finalizar, debe presentar los proyectos seleccionados por la mejor solución encontrada.

Punto 5. Experimentación y análisis

[1,0 puntos]

Ejecute el algoritmo al menos **cinco veces** con la configuración inicial. Posteriormente, realice los siguientes experimentos:

Experimento	Población	Generaciones	Mutación
A	10	50	0,01
B	20	100	0,05
C	50	200	0,10

Para cada experimento registre el mejor beneficio, el costo, los proyectos seleccionados, la generación en la que apareció la mejor solución, el tiempo de ejecución y la aptitud promedio de la última generación.

Construya una gráfica que muestre la mejor aptitud y la aptitud promedio por generación. A partir de los resultados, responda:

- a) ¿Qué efecto tuvo el tamaño de la población?
- b) ¿Cómo afectó la tasa de mutación a la diversidad?
- c) ¿Una tasa de mutación alta siempre produjo mejores resultados?
- d) ¿El mejor individuo apareció antes de la última generación?
- e) ¿El algoritmo obtuvo siempre la misma solución? Explique.
- f) ¿Qué evidencia indicaría una convergencia prematura?
- g) ¿Qué configuración recomendaría y por qué?

Requisitos técnicos

El programa debe incluir, como mínimo, las siguientes funciones:

```
generar_individuo()
generar_poblacion()
calcular_costo()
calcular_beneficio()
calcular_aptitud()
seleccionar_padre()
cruzar()
mutar()
ejecutar_algoritmo_genetico()
```

Se recomienda utilizar:

```
import random
import time
import matplotlib.pyplot as plt

# Semilla para experimentos reproducibles
random.seed(42)
```

No se permite utilizar bibliotecas que implementen directamente algoritmos genéticos. Las bibliotecas auxiliares para crear tablas y gráficas sí están permitidas.

Productos que se deben entregar

1. Código fuente en Python debidamente comentado.
2. Informe breve en PDF con la formulación y el diseño del algoritmo.
3. Tabla de resultados experimentales y gráficas de evolución.
4. Explicación de la mejor solución encontrada.
5. Conclusiones sobre la influencia de los parámetros.
6. Archivo `README.md` con instrucciones de ejecución.

Orientaciones finales

El algoritmo genético es una técnica estocástica; por lo tanto, una sola ejecución no es suficiente para evaluar su desempeño. Documente las decisiones de diseño, conserve evidencias de los experimentos y justifique las conclusiones utilizando los resultados obtenidos.